

Recursion vs Iteration: Activity-Sensitive differences across Use–Modify–Create Tasks in CS1

Ellen Yue Daetz
IT University of Copenhagen
CCER: Center for
Computing Education Research
Copenhagen, Denmark
ellen.daetz@hotmail.com

Louise Meier Carlsen
IT University of Copenhagen
CCER: Center for
Computing Education Research
Copenhagen, Denmark
loca@itu.dk

Claus Brabrand
IT University of Copenhagen
CCER: Center for
Computing Education Research
Copenhagen, Denmark
brabrand@itu.dk

ABSTRACT

Many studies have established that recursion is a notoriously difficult topic for CS1 students. We report on a quantitative controlled experiment involving $N=100$ CS1 students that characterizes student performance with linear recursion versus iteration across progressively more demanding programming activities defined by the Use–Modify–Create (UMC) framework. The study examines whether the order in which students practice the same programming tasks using iteration and linear recursion affects subsequent performance (RQ1), and compares performance using linear recursion versus iteration in terms of correctness (RQ2) and time-on-task (RQ3). Across activities, we find no meaningful effects of practice order, but observe a persistent correctness gap of approximately 10–20% across Use, Modify, and Create activities and approximately 40–50% longer time-on-task for linear recursion on active programming tasks (Modify and Create). These findings provide an empirical characterization of how CS1 students perform with linear recursion versus iteration under a conventional imperative-first curriculum, and clarify patterns of student proficiency that subsequent computing courses may rely upon.

CCS CONCEPTS

• **Social and professional topics** → **Computing education.**

KEYWORDS

Teaching/learning activities, student-learning perspective, recursion, iteration, Use-Modify-Create Framework

ACM Reference Format:

Ellen Yue Daetz, Louise Meier Carlsen, and Claus Brabrand. 2026. Recursion vs Iteration: Activity-Sensitive differences across Use–Modify–Create Tasks in CS1. In *Proceedings of the ACM Conference on International Computing Education Research Vol. 1 (ICER 2026 Vol. 1), August 11–14, 2026, Uppsala, Sweden*. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3765964.3811655>

1 INTRODUCTION

Recursion is an important programming technique to learn as part of the CS1 curriculum and is explicitly listed on the joint ACM/IEEE

Computing Curricula (CS2023) [2]. However, recursion is also notoriously difficult to learn [6, 16, 18, 25, 34]. Despite decades of research and pedagogical attention, recursion continues to be perceived as a challenging topic [12]. (We elaborate on this later.) In fact, recursion has been proposed as one out of eleven so-called *threshold concepts* that account for student difficulties in programming [19]. Aside from being important for introductory programming (CS1) in itself, recursion is also important for many later computing courses that rely on student competence in computation over inherently recursively defined structures, including functional programming, algorithms, semantics, and compilers. If CS1 fails to facilitate students’ acquisition of sufficient prerequisite competence in recursion, subsequent courses may be compromised; or, at least, rendered less effective. Many of these aim to train students in core computing competencies such as abstraction, problem decomposition, reasoning over recursive data structures, and functional programming.

Curricular Context & Scope. This study characterizes student performance with iteration and linear recursion within a conventional imperative-first CS1 curriculum. In this widely used instructional setting, iteration is introduced early and practiced extensively over several weeks, whereas recursion is introduced later and practiced more briefly. Rather than treating this instructional asymmetry as a confound to be eliminated, we treat it as a defining feature of the educational context under study and analyze how students perform under these realistic curricular conditions.

Study. We ran a quantitative controlled experiment involving $N=100$ CS1 students intended to compare recursion and iteration across three different types of activities from the UMC framework [22]. First, we compare how students fare on the same programming exercise problems using *first recursion, then iteration* vs *first iteration, then recursion* (RQ1). This examines whether the order in which students practice the two programming techniques on the same problems affects subsequent performance. We also analyse solving the same tasks on recursion and iteration in terms of *correctness* (RQ2) and *time-on-task* (RQ3), across the UMC Framework. This will give insights into the similarities and differences of how students fare with recursion and iteration.

Epistemic Stance. This study is descriptive rather than prescriptive. Our goal is to characterize how CS1 students perform when using iteration and linear recursion under a conventional imperative-first curriculum, not to argue for a particular teaching order, pedagogical intervention, or curricular reform. Accordingly, our findings should be interpreted as empirical characterizations

Book title	Iteration	Recursion	Ref
“Building Java Programs”	14% (2/14)	86% (12/14)	[32]
“Java Foundations”	19% (3/16)	100% (16/16)	[23]
“Java: How to Program”	20% (5/25)	72% (18/25)	[8]
“Java, Java, Java: OO...”	38% (6/16)	75% (12/16)	[31]
“Java Software Solutions”	38% (5/13)	92% (12/13)	[24]
Average	26% (~1/4)	85% (~5/6)	[–]

Figure 1: Chapter positions at which widely used Java programming textbooks first introduce iteration and recursion. (Percentages indicate relative position within the textbook.)

of observed performance patterns within this educational context, rather than as causal claims about how recursion ought to be taught.

Contributions. Ultimately, our study contributes insights into the proficiency of CS1 students using recursion and iteration across different activity types. This will help inform not only CS1 educators and educational programme managers who are responsible for sufficient student learning (here, competence in recursion), in general, but also educators of subsequent courses, who expectedly rely on prerequisite competences students ought to have acquired in CS1. According to fairly recent research, studies of CS1 students on basic programming tasks generalize to (industry) novice programmers [10]. Hence, our study also provides context for understanding relative differences in difficulty between recursion and iteration for novices, while remaining grounded in a CS1 educational setting.

2 BACKGROUND

We visit a few important background topics relevant for reading this paper. For studies aimed at comparing *recursion vs iteration*, we refer to Related Work (Section 3).

USE-MODIFY-CREATE (UMC) is a common pedagogical framework in Computing education [22] for organizing tasks into a hierarchy of progressively more demanding programming activities:

$$\text{USE} < \text{MODIFY} < \text{CREATE}$$

In this study, we operationalize the UMC framework primarily as three progressively more demanding categories of programming activity suitable for experimental comparison. First, students learn how to merely USE a piece of code (e.g., a function), including making predictions about it. Then, students progress into learning how to MODIFY the function so as to make it do something slightly different. Finally, students progress into the ability to CREATE an entirely new function. UMC also provides progression in terms of *code ownership* in that students first work with someone else’s code (in USE), then making the code partly their own (in MODIFY), and finally making their own code (in CREATE). The PRIMM approach sub-divides the USE activity into three sub-activities known as PREDICT, RUN, and INVESTIGATE (the first three letters of PRIMM),¹ and refers to the three UMC-activities as [35]:

$$\text{Not mine} < \text{Partly mine} < \text{All mine}$$

We will be working with PREDICT, MODIFY, & CREATE (as in PRIMM).

CS1 teaching Iteration & Recursion. In CS1 courses based on the imperative (and object-oriented) paradigm, iteration is usually

taught early, whereas recursion is typically covered late. Figure 1, shows a breakdown of when common text books² teaching Java programming and recursion introduce iteration, respectively, recursion in terms of chapter numbers. We see that iteration is generally introduced early on (on average, during the first quarter of the textbook), whereas recursion is generally introduced towards the end (on average, during the last sixth of the book).

Linear Recursion [1, §1.2.1] is a subset of recursion expressible as a single recursive function within the following restrictions:

- (1) has a clear *base case* effectively termination the recursion (e.g., when the argument becomes zero or the empty string);
- (2) makes only *one recursive call* (per call, in its body); and
- (3) calls recursively only on *smaller problems* in one direction (e.g., lower number or shorter string).

This encompasses many common recursive problems such as factorial, summation of numbers in a list, and palindromic string identification, but not Fibonacci, Towers of Hanoi, nor arithmetic expression evaluation that all require making multiple recursive calls (per call, in their function bodies); aka, tree recursion [1, §1.2.2]. Our study takes place in the context of simple, linear recursion.

Recursion Challenges. The literature reports a number of challenges regarding teaching and learning how to program using recursion. According to the literature, control flow is a challenge; in particular, the so-called “passive flow” upon return from recursive calls once the program reaches the base case (effectively terminating the recursion) [14, 21, 34]. Difficulties presumably lie in the resolution of the many pending, unfinished recursive calls on the call stack in reverse order of the calls made.³ The literature also mentions that data flow [29] and tracking local state [13], appear to challenge novice programmers. Recent work has additionally examined how students conceptually frame and reason about recursion through metaphor analysis [17]. Collectively, the challenges contribute to the widespread belief (especially among students) that:

“recursion is difficult”

in particular, that it is “more so than iteration” which is the topic of our study. We now zoom in on related studies that compare iteration vs recursion.

3 RELATED WORK

We focus related work on studies comparing recursion vs iteration and divide them according to evidence of a favored technique:

Studies favoring Recursion. In 1996, Benander et al. ran a three-year code comprehension (i.e., USE) experiment where $N=275$ students were randomly given either iterative or recursive (PASCAL) code manipulating linked lists: (1) searching the list for a number; and (2) copying the list [6]. Students were asked to write a single-sentence description of the purpose of the code (grades by instructors). For both programs, recursion was faster. In terms of correctness, searching a linked list favored recursion, whereas copying a linked list favored iteration, but insignificantly so.

Studies favoring Iteration. A decade earlier (in 1986), Kessler & Anderson compared student ability to write recursive vs iterative

¹The two last letters abbreviate MODIFY and MAKE (synonymous with CREATE).

²We ignore (popular) books where recursion is added only in later editions; e.g., [5].

³With the exception of tail recursion, which behaves like (stateless) “goto” statements.

Favors	Summary & Context	Year	Ref
Recursion	Recursion led to faster and more accurate comprehension for linked list <i>searching</i> (but not <i>copying</i>).	1996	[6]
Inconclusive	Prior work with iterative code may facilitate better understanding of subsequent recursive version.	1988	[37]
Inconclusive	Students taught iteration before recursion vs recursion before iteration performed equally well.	2012	[27]
Inconclusive	Students <i>prefer</i> iteration over recursion, but their solutions are not necessarily more correct.	2016	[36]
Inconclusive	Understanding simple iterative vs recursive code was similar wrt. both correctness & response time.	2022	[4]
Iteration	Students training on recursion made significantly more errors than those training on iteration.	1986	[20]
Iteration	For linked list <i>copying</i> , iteration is better (for <i>searching</i> , neither recursion nor iteration is better).	2015	[26]
Iteration	On CS2 tree problems, iterative solutions were more correct than recursive or mixed solutions.	2018	[9]

Figure 2: Evidence from prior studies comparing *recursion* and *iteration*.

functions in the context of SIMPLE, a LISP-like “close to English” language [20]. They found positive transfer from writing iterative functions to writing recursive functions, but not vice versa. “In the training phase, the subjects working with the recursive construct made significantly more errors than the subjects working with the iterative construct (32 vs 18 errors)” [20, p. 145].

McCauley et al. [26] replicated Benander et al.’s earlier study (mentioned above). For copying a linked list, the results were confirmed (favoring iteration). For searching a linked list, however, the results were not: Contradictionally, this later study found that students were equally likely to correctly describe the purpose of the search algorithm, whether iterative or recursive.

Esteero et al., ran a CS2 course where students could choose between recursion vs iteration [9]. They found that for a tree-based exam question, a fifth of students (19%) choose to use iteration, half (51%) choose recursion, and a sixth (16%) choose to combine both iteration & recursion. They also found that students choosing iteration perform better than those who chose recursion and the combined technique.

Inconclusive Studies. In 1988, Wiedenbeck carried out two controlled (Use) experiments with $N=116$ students comparing already given recursive solutions vs iterative ones and found that: For recursive mathematical specifications, “prior experience with similar iterative problems was not an aid [and vice versa]” [37, p. 278]. For recursive code implementations, “prior work with the iterative version of a program may facilitate understanding of the recursive version, at least to some extent [but not vice versa].” [37, p. 278].

In 2012, Mirolo compared students exposed to a functional-first (introducing recursion before iteration) vs imperative-first (iteration before recursion) courses [27], over three years. The results show that students performed equally well overall, indicating no evidence for teaching iteration before recursion (as common in most CS1).

In 2016, Sulov conducted an experiment involving $N=130$ novice programming students exposed to tasks solvable by both recursion & iteration. [36]. The results showed students strongly preferred iteration over recursion even on tasks that would naturally favor recursion; e.g., Fibonacci, generate all permutations, and directory traversal. The many students who chose iteration do not necessarily perform better than the fewer ones who chose recursion. After exposure to correct iterative and recursive solutions, many students changed their preferences.

More recently, an eye-tracking experiment had $N=117$ students reading simple recursive vs iteration code [4]. They found no difference in understanding simple iterative vs recursive code, neither in terms of correctness, nor in response time. Also, they found that students follow a comparable reading behavior for both iterative and recursive programs, in terms of visual attention.

Research gap. In summary, one older study shows evidence favoring recursion over iteration, but for a Use task on an inherently recursive data structure (searching a linked list). Three studies favor iteration over recursion, but in specific contexts: for a “close to English” (LISP like) language, for recursive data structures, and for tree-recursion in CS2. Most studies, however, remain inconclusive. Our intention is to “zoom in” and study simple, linear problems that have concise 4–7 line canonical iterative & recursive solutions, in the context of imperative-first CS1. By “zooming in”, we utilise the Use, Modify, Create framework to consider different type tasks with different complexity levels.

4 METHODOLOGY

We now describe our controlled quantitative experimental setup.

4.1 Objectives

We consider the following research questions centered around comparing *iteration vs recursion*. Since programming is comprised of many different kinds of activities, we deliberately invoke the UMC Framework [22] so as to investigate for progressively more complex programming activities separately. The first research question addresses whether the order of practice between recursion and iteration affects student performance when solving the same programming problems. For this, we investigate the *sequencing* of the two techniques, importantly, on the same programming problems:

RQ1 (SEQUENCING): How effective is it for CS1 students to solve the same Use/MODIFY/CREATE programming problems using *first iteration, then recursion* vs *first recursion, then iteration*?

We also study the effectiveness of CS1 students solving programming tasks using recursion compared to using iteration (in terms of *correctness & time-on-task*):

RQ2 (CORRECTNESS): How do CS1 students’ performance differ and compare between *iteration* and *recursion* in terms of *correctness* for Use, MODIFY, and CREATE tasks, both at the aggregate level and

across individual performance?

RQ3 (TIME-ON-TASK): How do CS1 students' performance differ and compare between *iteration* and *recursion* in terms of *time-on-task* for USE, MODIFY, and CREATE tasks, both at the aggregate level and across individual performance?

Implicit above, we consider only simple, *linear recursion* problems.

4.2 Educational Context

This experiment takes place in the context of the CS1 course which is a mandatory 15 ECTS⁴ first-semester course on the Bachelor of Software Development (CS majors) at the IT University of Copenhagen. The course is divided into two parts: The first 12 weeks of the course teaches imperative and object-oriented programming in Java using a “*zipped teaching format*” (interleaved lectures-&-exercises) [7]. Iteration is taught at the beginning of the course (session 2, week 1), while recursion is taught towards the end (session 22, week 11), which is a common structure (cf. Figure 1). Our experiment took place in the teaching/learning session following the one that introduced recursion (session 23 in week 12 of the course). The last 3–4 weeks of the course are dedicated to a student group project undertaken in groups of three. This is a chance to practice the disciplinary skills that the students acquired during the first 12 weeks of the course on a practical project [30]. The course finishes with an individual oral final exam relative to explicitly defined intended learning outcomes.

4.3 Subject Selection

The experiment was conducted as a non-mandatory exercise on the topics recursion & iteration in the CS1 course during the Fall of 2024. CS1 students were invited to participate, representing novice programmers in the process of learning to program through formal education. Importantly, students could *anonymously opt out* of having their data used for research purposes (no one did). In 2024, a total of 154 students were enrolled in the CS1 course. The median age was 21 years and around 20% were women. Students without prior programming experience (around 3 in 10 of the men vs about half of the women) had been offered a 3-day elective programming onboarding course before the semester started. Prior research has shown that inexperienced students who attended this elective course effectively closed the competence gap associated with prior programming experience [15].

4.4 Treatment: Programming Technique

The **treatment** is the programming technique used for solving programming tasks: either *recursion* ($\boxed{R}$) xor *iteration* ($\boxed{I}$):

$$T = \{ \boxed{R}, \boxed{I} \}$$

We will effectively use iteration as a contrasting baseline or control treatment for studying the *experimental treatment*: recursion.

4.5 Programming Problems

We consider the following four programming problems:

$$\mathbb{P} = \{ P_0, P_1, P_2, P_3 \}$$

where:

- $P_0 = \text{fac}$: calculate factorial of n : $n!$
- $P_1 = \text{twoPow}$: calculate the powers-of-two of n : 2^n
- $P_2 = \text{countA}$: count #as in string s : $\{ 0 \leq i < |s| : a = s[i] \}$
- $P_3 = \text{reverse}$ string s : r where $r[i] = s[|s|-1-i]$ for $0 \leq i < |s|$

P_0 (fac): will serve as a demo problem (aka, training task) to show the participants what kind of exercises they will be exposed to and what answers are expected to look like. During the experiment, the subjects will be exposed to the real problems (aka, test tasks): $[P_1, P_2, P_3]$, in that order. These programming problems are common problems used in the books in Figure 1 either as demonstrations or exercises. All four problems considered can be solved as simple, linear iterative/recursive functions (cf. Section 2) [1, §1.2.1].

Canonical Solutions. Figure 3 provides a characterization of recursive vs iterative canonical solutions to the four problems P_0 – P_3 . Canonical implementations are provided in Figure 4. In terms of size (LOC), we see, as expected, that canonical recursive solutions are generally more concise than iterative ones. Since the programs are so short (4–7 lines), we also provide the number of characters as a fine-grained descriptive size measure. Again, we see that recursive solutions are generally more succinct than iterative ones (with the exception of the canonical recursive solution to P_2 which could be shortened. Also, recursive solutions usually do not need extra state beyond the arguments to the recursive calls (implicitly saved on the call stack); iterative solutions, on the other hand, need a for-loop along with explicit local state in the form of extra local variables to keep track of: (1) how far the iteration has gotten (iteration variable, i); and (2) the intermediate result (result variable, res). (This is why the #iteration column consistently says 0 vs 1, while the #Locals column consistently says 0 vs 2.) Recursive solutions usually have multiple return-statements along with a conditional if-statement (for discovering and handling recursive base cases such as $n=0$); iterative solutions, in turn, cope with trivial cases by vacuously “skipping over” the (for)-iteration entirely. In summary, we see that canonical recursive vs iterative solutions are quite different, as also testified by running the Moss code similarity (plagiarism) checker [33]; it claims all four program pairs are unsimilar (no matches found). (Keep this in mind for RQ1, later on.)

4.6 Programming Activity: USE-MODIFY-CREATE

Our experiment is structured around the UMC framework [22], based on a hierarchy of three progressively more demanding kinds of programming activities: $\underline{U}SE < \underline{M}ODIFY < \underline{C}REATE$. The set of programming **activities** is defined as:

$$\mathbb{A} = \{ \underline{U}, \underline{M}, \underline{C} \}$$

The task that subjects will be exposed to will thus be different for each of the three qualitatively different kinds of activities:

USE. Subjects are asked to predict the inputs & outputs (I/Os) of invoking a given recursive or iterative function with a specific argument. For recursion, the I/Os-context is the recursive function itself, so: inputs become the sequence of successive argument values (in all invocations to the recursive function); whereas outputs become the sequence of successive values returned (from all recursive calls). Figure 5a (top left) provides the correct answers for the recursive solution to the demo problem P_0 with argument $n=3$; i.e., $\text{fac}(3)$: inputs is the sequence of arguments to fac : $[3, 2, 1,$

⁴One academic year is 60 ECTS (European Credit Transfer and Accumulation System).

	Function		Size		Treatment		Control		State	R vs I
	Name	Signature	#Lines	#Chars	#recursion	#iteration	#if	#return	#Locals	Similarity
P ₀	fac	int → int	4 vs 6	47 vs 63	<u>1</u> vs 0	0 vs <u>1</u>	1 vs 0	2 vs 1	0 vs 2	unsimilar
P ₁	twoPow	int → int	4 vs 6	53 vs 65	<u>1</u> vs 0	0 vs <u>1</u>	1 vs 0	2 vs 1	0 vs 2	unsimilar
P ₂	countA	Str → int	6 vs 7	140* v 106	<u>2</u> * v 0	0 vs <u>1</u>	2 vs 1	3 vs 1	0 vs 2	unsimilar
P ₃	reverse	Str → Str	4 vs 6	92 vs 95	<u>1</u> vs 0	0 vs <u>1</u>	1 vs 0	2 vs 1	0 vs 2	unsimilar

Figure 3: Characterization of canonical solutions to the problems: { P₀, P₁, P₂, P₃ } (P₀ for *training* & P_{1–3} for *testing*); function name & type signature, #Lines (LOC) & #Characters (without spaces), #recursive function invocations & #iterative for loops, #if-statements & return-statements, and #local variable declarations. (Metrics are given as *recursion* vs *iteration*, if applicable). The rightmost column, gives the results of running a code similarity (plagiarism) checker on the *recursive* vs *iterative* solution. (*Instead of lexically having two identical recursive calls; then, make only one shared recursive call, store the result in a local variable, and read its value twice: This would yield 122 rather than 144 characters and 1 rather than 2 recursive calls.)

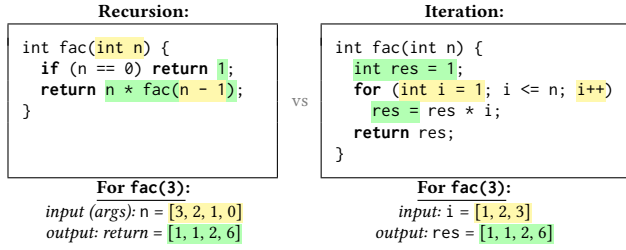
P#	Recursion	Iteration
P ₀ :	<pre>int fac(int n) { if (n == 0) return 1; return n * fac(n - 1); }</pre>	<pre>int fac(int n) { int res = 1; for (int i = 1; i <= n; i++) res = res * i; return res; }</pre>
P ₁ :	<pre>int twoPow(int n) { if (n == 0) return 1; return 2 * twoPow(n - 1); }</pre>	<pre>int twoPow(int n) { int res = 1; for (int i = 0; i < n; i++) res = res * 2; return res; }</pre>
P ₂ :	<pre>int countA(String word) { if (word.length() == 0) return 0; if (word.charAt(0) == 'a') return 1 + countA(word.substring(1)); return countA(word.substring(1)); }</pre>	<pre>int countA(String word) { int res = 0; for (int i = 0; i < word.length(); i++) if (word.charAt(i) == 'a') res = res + 1; return res; }</pre>
P ₃ :	<pre>String reverse(String s) { if (s.length() == 0) return ""; return reverse(s.substring(1)) + s.charAt(0); }</pre>	<pre>String reverse(String s) { String res = ""; for (int i = 0; i < s.length(); i++) res = s.charAt(i) + res; return res; }</pre>

Figure 4: Canonical solutions using *recursion* (left hand side, highlighted in yellow) vs *iteration* (right, highlighted in orange) for programming problems: { P₀, P₁, P₂, P₃ }. (In the code listed above, we have removed superfluous symbols such as '{' & '}').

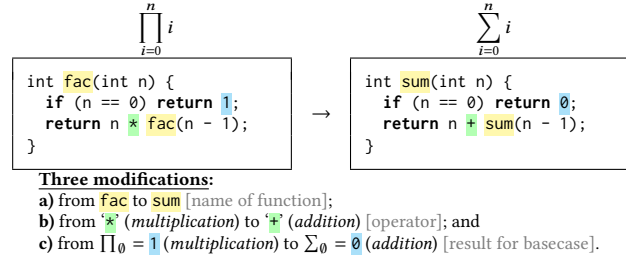
0]; whereas outputs is the sequence of all values returned: [1, 1, 2, 6]. For iteration, the I/Os-context is the for-statement (loop), so: inputs is interpreted as all successive values of the iteration variable, i; whereas outputs is taken as all successive values of the result variable, res. For the demo exercise in Figure 5a (top right), the correct answers are: [1, 2, 3] for inputs and [1, 1, 2, 6] for outputs.

MODIFY. Subjects are asked to slightly change what a given function does, as in reprogramming the function. Figure 5b gives

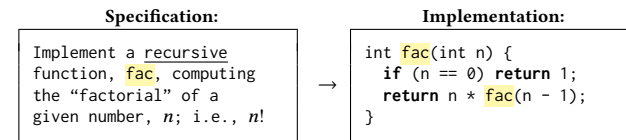
the example exercises for demo problem P₀ for recursion. Here, the task is to change from multiplying the numbers 0–n to summing the numbers 0–n. This involves three modifications: (i) changing the name from *fac* to *sum*; (ii) the operator from *** to *+*; and (iii) the neutral element from 1 (for multiplication) to 0 (for addition) in the base case. (The iteration activity for P₁ is similar and simply involves modifying an iterative instead of recursive function; not shown in Figure 5.)



(a) Use using recursion (left) vs iteration (right).



(b) Modify w/ recursion: before (left) vs after (right) modification.



(c) Create w/ recursion: specification (left) vs implementation (right).

Figure 5: Demo exercise P_0 (fac) as { USE, MODIFY, CREATE }.

CREATE. Subjects are tasked with implementing a function from scratch based on a specification. Figure 5c illustrates this for demo problem P_0 for recursion. (Again, the iteration activity is similar; the solution will simply use iteration rather than recursion; the solution can be seen in Figure 4 (top right).)

Assessing Solutions. Use activities are assessed by (objective) multiple-choice questions with exactly one correct answer along with several distractors [11]. Active programming activities (Modify & Create) were assessed by the first author using a rubric based on the explicit criteria outlined in Figure 6, thereby minimizing subjectivity in the code assessment [3]. One point is awarded for each of the six criteria for correct recursive, respectively, iterative solutions. Results are quantified as ratios (or, equivalently, percentages) as in, e.g., “3 out of 6” (or, 50%).

4.7 Programming Exercise

A programming exercise, $X \in \mathbb{X} = \mathbb{P} \times \mathbb{A} \times \mathbb{T}$, is composed of a problem, an activity, and a treatment:

$$\mathbb{X} = \{P_0, P_1, P_2, P_3\} \times \{\underline{U}, \underline{M}, \underline{C}\} \times \{\underline{R}, \underline{I}\}$$

Thus, there are $|\mathbb{X}| = 4 \times 3 \times 2 = 24$ distinct exercises, in total; Figure 5 showcases four of these, namely the following exercises (where $P_1 = \text{twoPow}$):

$$\begin{aligned} &\langle P_1, \underline{U}, \underline{R} \rangle \text{ (top left)} && \langle P_1, \underline{U}, \underline{I} \rangle \text{ (top right)} \\ &\langle P_1, \underline{M}, \underline{R} \rangle \text{ (middle)} \\ &\langle P_1, \underline{C}, \underline{R} \rangle \text{ (bottom)} \end{aligned}$$

4.8 Crossover Study (of Learning Transfer)

For RQ1, we perform a randomized controlled crossover (within-subjects) experimental design to study the learning transfer between solving the same exercise problems using recursion $\Leftrightarrow$ iteration. Here, subjects receive a temporal sequence of different treatments and act as their own control. We can thus quantify how an individual participant fares on the same exercise problem using first one of the programming techniques (either recursion or iteration) in comparison to how well they subsequently perform on the same exercise using “the other technique.” This amounts to studying the sequencing of the two techniques, by introducing two symmetric experimental *conditions*:

$$\mathbb{C} = \{ \underline{RI}, \underline{IR} \}$$

where:

$$\begin{aligned} \underline{RI} &= \text{first } \underline{R} \text{ecursion, then } \underline{I} \text{teration} \\ \underline{IR} &= \text{first } \underline{I} \text{teration, then } \underline{R} \text{ecursion} \end{aligned}$$

We will write $\underline{R}$ for recursion (after iteration) and $\underline{I}$ for iteration (before recursion). Hence, $\langle P_1, \underline{U}, \underline{R} \rangle$, e.g., will designate an exercise involving problem P_1 as a USE activity using Recursion, after having been exposed to the same exercise using Iteration: $\langle P_1, \underline{U}, \underline{I} \rangle$. (Similarly, for Iteration $\underline{I}$ after Recursion $\underline{R}$.) Half of the subjects will be randomly assigned to one condition, the remaining half to the other condition.

4.9 Latin Square (for Studying a Technique)

As detailed in RQ2 & RQ3, our objective is to study recursion vs iteration for each programming activity $A = \{\underline{U}, \underline{M}, \underline{C}\}$. We will use a randomized controlled between-subjects experimental design. Since there are three activities ($|\mathbb{A}| = 3$), we also need three problems in order to use a so-called **Latin Square** experimental design. Figure 7 illustrates how a Latin Square enables studying all combinations of problems & activities. Each subject is assigned one of the three colors (shades of gray) and will be exposed to each of the three problems as well as each of the three activities. According to the illustration, a subject assigned the mid-gray color (■), would, for instance, receive: problem P_2 as a USE exercise, P_1 as MODIFY, and P_3 as CREATE. This design accounts for learning bias by avoiding repeated exposure to the same problem and activity. Since the three activities comprising the UMC framework form a progression hierarchy, we insist on ordering exposure to always first USE, then MODIFY, and always last: CREATE. Accordingly, the experimental balancing concerns the assignment of problems and programming techniques to activities, rather than permuting the pedagogically natural UMC progression itself (Use $\rightarrow$ Modify $\rightarrow$ Create).

For this design, we can use one Latin Square for studying recursion and another for studying iteration. Figure 8 illustrates the entire experimental design. Subjects assigned to the $\underline{RI}$ condition traverse the figure left-to-right; those assigned to $\underline{IR}$, right-to-left.

Recursion	Iteration
<pre>int fac(int n) { if (n == 0) return 1; return n * fac(n - 1); }</pre>	<pre>int fac(int n) { int res = 1; for (int i = 1; i <= n; i++) res = res * i; return res; }</pre>
- control structures - base case condition - base case result - recursive decomposition - recursive call - recursive recomposition	- control structures - result computation - result initialization & return - loop-variable initialization - loop-variable condition - loop-variable incrementation

Figure 6: Rubric grading criteria for active programming activities (MODIFY & CREATE), illustrated for demo task P_0 .

	U	M	C
P ₁			
P ₂			
P ₃			

Figure 7: Latin Square (3×3) design for studying programming problems × activities: {P₁, P₂, P₃} × {Use, Modify, Create}. A subject is randomly assigned a color and is exposed to 3 corresponding problem-&-activity exercise combinations.

	U	M	C
P ₁	R	R	R
P ₂	R	R	R
P ₃	R	R	R
Recursion			

	U	M	C
P ₁	I	I	I
P ₂	I	I	I
P ₃	I	I	I
Iteration			

RI ⇌ IR

Figure 8: Two conditions (sequences of exposure): $\boxed{\text{RI}}$ (first recursion, then iteration) & $\boxed{\text{IR}}$ (first iteration, then recursion). Half of the subjects are assigned to $\boxed{\text{RI}}$, the other half to $\boxed{\text{IR}}$.

4.10 Analysis

For statistical analysis, we will use the Mann-Whitney U-test⁵ (aka, Wilcoxon Rank-Sum) to compare the vectorized data. For comparing two proportions, we will use the Z-test⁶ for two population proportions. As is customary, we adopt a 95% confidence interval (i.e., $\alpha=5\%$) and use two-tailed tests, asymmetrically not presupposing the superiority of either testee.

5 RESULTS

N=100 students participated in the aforementioned experiment when the topic of the CS1 course became recursion (in November of 2024). This represents 71% (100 out of 141) of the active students (taken as those who eventually completed all mandatory course requirements).

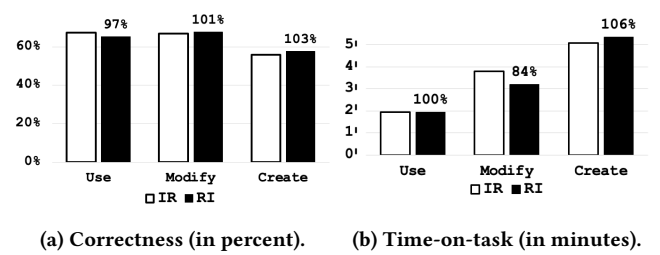


Figure 9: Results for exercises under first-iteration-then-recursion ($\boxed{\text{IR}}$, in white) versus first-recursion-then-iteration ($\boxed{\text{RI}}$, in black), according to activity: {U, M, C}. The $\boxed{\text{RI}}$ percentages are relative to a 100%-normalized $\boxed{\text{IR}}$ baseline.

5.1 Sequencing of {IR, RI} × {U, M, C} (RQ1)

Correctness. Figure 9a shows correctness for exercises completed under $\boxed{\text{IR}}$ (white) versus $\boxed{\text{RI}}$ (black) for each activity in the UMC framework. Across all activities, mean correctness scores cluster around 60%. Differences between the two practice orders are small, remaining within ± 3 percentage points (97%–103% relative to baseline), and none are statistically significant ($p = 0.59$ for $\boxed{\text{USE}}$, $p = 0.94$ for $\boxed{\text{MODIFY}}$, and $p = 0.76$ for $\boxed{\text{CREATE}}$).

Time-on-task. Figure 9b reports time-on-task for $\boxed{\text{IR}}$ versus $\boxed{\text{RI}}$. To reduce noise from premature abandonment, students who submitted within 20 seconds (12 students in total) were excluded from the averages. As expected, mean time-on-task increases along the UMC progression: approximately two minutes for $\boxed{\text{USE}}$, 3–3½ minutes for $\boxed{\text{MODIFY}}$, and around five minutes for $\boxed{\text{CREATE}}$. Differences between practice orders are modest, within ± 16 percentage points (84%–106%), and again not statistically significant ($p = 0.46$ for $\boxed{\text{USE}}$, $p = 0.054$ for $\boxed{\text{MODIFY}}$, and $p = 0.44$ for $\boxed{\text{CREATE}}$).

Summary. Across all three activities (U, M, C), we observe no statistically significant differences in correctness or time-on-task between the two practice orders ($\boxed{\text{IR}}$ versus $\boxed{\text{RI}}$). The same pattern holds when considering only recursion (recursion practiced first versus last) or only iteration (iteration practiced first versus last).

OBSERVATION 1 (Practice-Order Insensitivity). Under a conventional imperative-first CS1 curriculum,

⁵U-test: <https://www.socscistatistics.com/tests/mannwhitney/>

⁶Z-test: <https://www.socscistatistics.com/tests/ztest/>

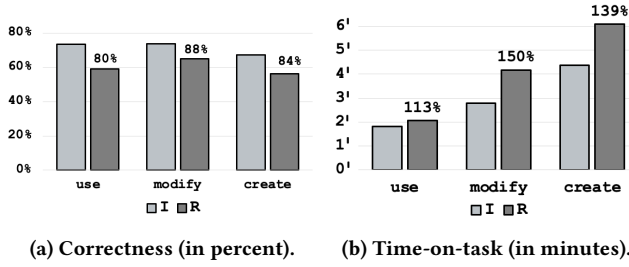


Figure 10: Results for exercises under iteration (I) light gray) versus linear recursion (R dark), according to activity: {U, M, C}. The R percentages are relative to a 100%-normalized I baseline.

students' correctness and time-on-task when solving a programming task using iteration or linear recursion do not differ meaningfully depending on whether the other technique was practiced first, even when the tasks were identical.

One possible explanation for this practice-order insensitivity is that the canonical solutions for iteration and linear recursion differ substantially in structure, as illustrated in Figures 3 and 4. For the remainder of the paper, we therefore compare recursion and iteration irrespective of practice order, collating:

$$\boxed{R} = \boxed{R-} \cup \boxed{R+} \quad \boxed{I} = \boxed{I-} \cup \boxed{I+}$$

Our results are consistent with Mirolo [27], who found no overall performance difference between functional-first and imperative-first instruction. They contrast with the earlier findings of Kessler and Anderson [20], who reported positive effects of prior iteration practice on recursion in a Lisp-like language.

5.2 Correctness of {I, R} × {U, M, C} (RQ2)

Figure 10a shows correctness results for iteration (I, light gray) versus linear recursion (R, dark gray) for each of the activities {U, M, C}. Across all activities, correctness under linear recursion is consistently lower than under iteration, with relative differences in the range of approximately 10–20 percentage points. Statistical analysis confirms that these differences are statistically significant for all three activities ($p = 0.00157$ for USE, $p = 0.00291$ for MODIFY, and $p = 0.00192$ for CREATE; aggregate $p < 10^{-6}$).

OBSERVATION 2 (Persistent Correctness Gap): Under a conventional imperative-first CS1 curriculum, students are consistently less correct when solving USE, MODIFY, and CREATE tasks using linear recursion than iteration, with a gap of approximately 10–20 percentage points across activity types.

Distribution. To examine how these aggregate differences arise at the individual level, Figure 11a presents the distribution of correctness scores for linear recursion versus iteration across activities. Already at the level of USE, the two techniques exhibit distinct distributional patterns: iteration shows a steadily increasing concentration toward higher correctness scores, whereas linear recursion displays a vague U-shaped distribution, with a high number

of students either solving the task entirely correctly or struggling substantially. For MODIFY, iteration begins to exhibit a weak U-shaped distribution, with a local minimum around 2/6 and a strong concentration of students between 4/6 and 6/6, indicating that most students either solve the task completely or exhibit only minor misunderstandings. Linear recursion shows a similar overall shape but with two high peaks showing that most frequently, students solve the task correctly or with two misunderstandings. For CREATE, both techniques exhibit clearly pronounced U-shaped distributions, indicating increased polarization in performance for more demanding programming tasks. This suggests that as task demands increase, student performance becomes increasingly bimodal, with fewer students achieving partial correctness and more clustering at the extremes.

Relative correctness ratio. Because each student solved tasks using both techniques, we further compute a paired relative correctness ratio (recursion relative to iteration) for each participant. Figure 11b shows the distribution of these ratios on a logarithmic scale. For USE, the majority of observations lie to the left of parity (i.e., ratios below 1), with frequencies increasing steadily toward the center, while the right-hand side exhibits a more rapidly curving decline. This asymmetric shape indicates that small-to-moderate performance disadvantages for linear recursion are more common than corresponding advantages. For MODIFY, the distribution differs markedly in shape, with a higher and sharper peak around parity and more irregular (“squiggly”) variation on both sides. The difference between the left and right tails is less pronounced than for USE, though the distribution remains slightly heavier on the side favoring iteration. For CREATE, the distribution again resembles that of USE, with a lower central peak, smoother tails, a clearly heavier left side, and a more sharply curving right-hand slope. This pattern may indicate that performance disadvantages for linear recursion are more visibly pronounced for the least and most demanding activities, whereas modification tasks appear more heterogeneous and somewhat closer to parity for a subset of students

5.3 Time-on-Task of {I, R} × {U, M, C} (RQ3)

Please note that time-on-task should not be confused with the run-time performance of the implemented programs. For example, in Java, the iterative version of P_0 expectedly executes faster than the recursive version. Moreover, for tail-recursive programs and languages with tail-call optimization, such differences may be reduced due to elimination of stack-frame overhead. Our focus here is exclusively on the time students spend solving the tasks.

Figure 10b shows time-on-task with outliers excluded (submissions ≤ 20 seconds, as before) for iteration (I, light gray) versus linear recursion (R, dark gray) across activities {U, M, C}. Across all activities, students spend more time when solving tasks using linear recursion than iteration. On average, time-on-task is approximately 13% higher for USE activities, 50% higher for MODIFY, and 39% higher for CREATE tasks. These differences are statistically significant ($p = 0.02095$ for USE, $p = 0.0004067$ for MODIFY, and $p = 0.01894$ for CREATE; aggregate $p = 0.00000707$).

OBSERVATION 3 (Time-on-Task Amplification):

Under a conventional imperative-first CS1 curriculum, students require systematically more time to

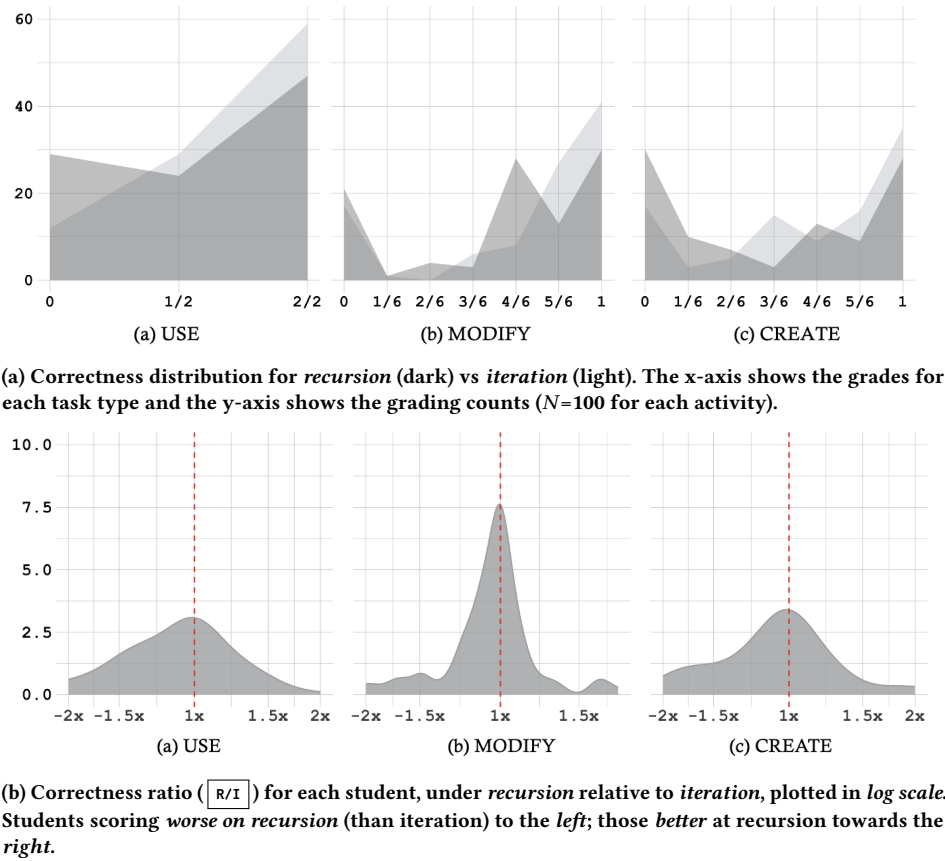


Figure 11: Correctness results (for RQ2) for each of the activities: USE (left), MODIFY (center), and CREATE (right).

solve tasks using linear recursion than iteration, with the relative time-on-task gap increasing for more active programming activities and reaching approximately 40–50 percentage points for MODIFY and CREATE tasks.

Distribution. Figure 12a shows the distribution of time-on-task for each activity. For USE, differences between iteration and linear recursion are modest. Both distributions exhibit a pronounced peak at very short durations (approximately 0–2 minutes), followed by a rapid decline and very few occurrences beyond six minutes, indicating that most students complete these tasks quickly regardless of technique. For MODIFY, the distributions diverge more clearly. Under iteration, the distribution exhibits a high and sharp peak around two minutes, with a steep incline and relatively short tail, suggesting that many students complete these tasks efficiently. In contrast, linear recursion shows a noticeably lower peak occurring around three minutes, accompanied by a broader and longer tail, indicating greater variability and a higher proportion of students spending substantially more time on the technique. For CREATE, the difference in distributions is most pronounced. Although both techniques peak around similar central values, the peak for iteration is higher and followed by a steeper decline, whereas linear recursion

exhibits a lower peak and a consistently broader tail spanning approximately two to eight minutes. This pattern suggests that while many students complete create-level tasks within comparable time ranges under both techniques, linear recursion is associated with a larger subgroup of students who require substantially more time.

Relative time-on-task ratio. Because each student solved tasks using both techniques, we further examine paired relative time-on-task ratios (recursion relative to iteration). Figure 12b shows the distribution of these ratios on a logarithmic scale with outliers removed. For USE, the distribution exhibits the tallest peak around parity, along with the steepest rise and decline, indicating that most students spend comparable amounts of time under the two techniques for simpler tasks. In contrast, the distributions for MODIFY and CREATE are quite similar to each other, with lower peaks and broader tails. This flattening and widening of the distributions indicates increased heterogeneity in relative time-on-task for more demanding activities, with a larger proportion of students exhibiting substantial slowdowns under linear recursion relative to iteration. Together, these patterns suggest that while time differences between techniques are tightly clustered for simpler tasks, the relative cost of linear recursion becomes more variable—and for many students more pronounced—as task demands increase.

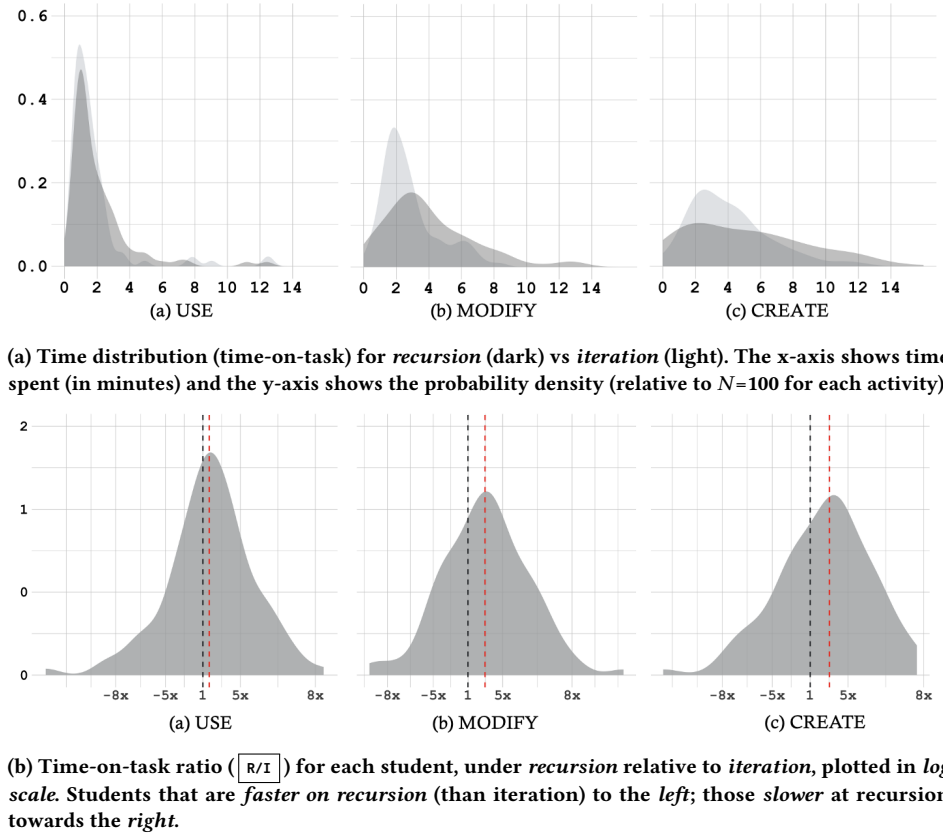


Figure 12: Time-on-task results (for RQ3) for each of the activities: USE (left), MODIFY (center), and CREATE (right).

6 THREATS TO VALIDITY

We discuss threats to the validity of our results with a focus on issues that could plausibly affect the interpretation of the observed differences between recursion and iteration.

Measurement of correctness and time-on-task. The USE activities of the experiment are, in fact, from the PREDICT sub-activity category (cf. Section 2). The tasks compare the I/O context which is the (recursive) function itself for recursion versus the (iterative) for-loop for iteration. Scoring is based on multiple-choice questions, which mitigate threats regarding subjective assessment.

For assessing code solutions produced under Modify & Create activities, we relied on a criterion-referenced assessment rubric for scoring programming solutions, to minimize grading subjectivity. Rubrics help reduce subjectivity by requiring that judgments be based on specific descriptions of performance rather than general impressions [3]. For recursion, there were six criteria; for iteration, there were four. This difference is mitigated by reporting all correctness scores as ratios or percentages; e.g., 3 out of 6 (50%) vs. 2 out of 4 (50%).

Exercise duration (time-on-task) was measured via the online system. While self-paced timing may include interruptions (e.g., short breaks or premature submission), such noise applies symmetrically to both recursion and iteration and is unlikely to explain the consistent differences observed across activities.

Task design and difficulty. There may be risks of ceiling (or floor) effects where many subjects achieve the maximum (or minimum) score limit such that differences attributable to treatment cannot be discerned. From Figure 10a, we see that average student correctness was around 50%–70%, which we take as an indication of appropriate difficulty for the exercises used in this experiment. Figure 11a further shows that the full correctness spectrum (0%–100%) is in use for both iteration and recursion.

To mitigate the risk that participants misunderstood the tasks, we ran a pilot experiment with CS1 teaching assistants, which led to revisions of task formulations and calibration of exercise difficulty. We also added a demonstration exercise so that participants knew what questions and answers looked like for each of the UMC activities. Moreover, the subjects had encountered similar exercise formats during the course, including multiple-choice questions, analytical tracing exercises, synthesis exercises, and USE, MODIFY, and CREATE activities. The experiment used the same programming language as the course (Java) and was conducted in Danish, the participants' native language.

Prior exposure, individual differences, and experimental control. Students had more training in iteration (introduced early in the course) than recursion (introduced later), which reflects a typical structure of imperative-first CS1 courses. This asymmetry

is therefore intrinsic to the educational context under study rather than an artifact of the experimental design.

Our experiment compensates for individual differences, including programming competence, through experimental design. Importantly, the same student was exposed to exercises using both recursion and iteration, allowing us to quantify the relational ratio between their ability to program with recursion versus iteration, irrespective of their individual (absolute) programming level. The randomized crossover and Latin Square designs further account for learning effects and ordering biases. With $N=100$ subjects, the influence of individual differences is expected to be reduced through averaging across participants and the within-subject experimental design [28].

To avoid stressing or biasing participants, all results were anonymized, and we explicitly stated that we were not interested in individual performance but only general patterns. The purpose of the study was not revealed to the students to avoid compromising experimental validity. Assessment relied on objective multiple-choice questions and a well-defined rubric, minimizing experimenter-expectancy bias.

Scope and generalizability. The findings reported in this study are intended to characterize student performance under a specific and common educational context. In particular, they apply to:

- **Subjects:** First-semester CS1 students in imperative-first, Java-based introductory programming courses;
- **Problems:** Simple programming problems solvable using linear recursion;
- **Tasks:** Short programming tasks with concise canonical solutions (4–7 lines of code);
- **Activities:** Activity types spanning Use, Modify, and Create as defined by the UMC framework.

The results do not directly generalize to more complex forms of recursion such as tree recursion (e.g., Towers of Hanoi or expression parsing), to computations over recursively defined data structures where recursion may be more natural than iteration, to functional-first curricula, or to more experienced student populations. These boundaries are intentional and align with the study’s objective of empirically characterizing how CS1 students perform with recursion and iteration within a conventional imperative-first introductory programming context.

In summary. The threats discussed above serve primarily to contextualize the scope and interpretation of our study. Importantly, none of the threats significantly compromise the integrity of the results or the central findings reported in this paper. While certain design choices and contextual factors necessarily bound the generalizability of our conclusions, the experimental setup, measurement approach, and effect sizes observed across activities provide consistent evidence for the reported differences between recursion and iteration for CS1 students.

7 CONCLUSION

We reported on a controlled experiment using a Latin-square and crossover design that characterized how CS1 students perform

when using iteration and linear recursion in a conventional imperative-first introductory programming course. The study examined practice-order sensitivity (RQ1), correctness differences (RQ2), and time-on-task differences (RQ3) across progressively more demanding programming activities defined by the Use–Modify–Create framework.

First (RQ1), with respect to practice order, we observed no meaningful differences in either correctness or time-on-task depending on whether students practised a given task using iteration or linear recursion first. This practice-order insensitivity held even when students solved identical tasks using both techniques, suggesting that short-term performance is robust to the sequencing of practice under the curricular conditions studied.

Second (RQ2), we found a persistent correctness gap between the two techniques. Across Use, Modify, and Create activities, students were consistently less correct when solving tasks using linear recursion than iteration, with relative differences of approximately 10–20 percentage points. This pattern was evident both at the aggregate level and in paired, per-student comparisons, and did not disappear as activity demands increased. We also saw that when the complexity level increased, the distributions converged towards a U-shape, showing polarisation of the scores. Interestingly, the paired correctness distributions of recursion and iteration showed that most students were equally correct at solving Modify, while Use and Create followed a flatter distribution.

Third (RQ3), we observed systematic differences in time-on-task. Students spent substantially more time solving tasks using linear recursion than iteration, with the relative gap increasing from Use to Modify and Create activities and reaching approximately 40–50% for active programming tasks. This trend indicates that the additional effort associated with linear recursion becomes more pronounced as tasks require greater code ownership and synthesis. Even when considering the relative distributions, we saw likewise that Use had the highest peak, while Modify and Create followed roughly the same broader distribution.

Our findings are broadly consistent with prior work reporting increased difficulty for recursion relative to iteration [9, 20]. At the same time, our results align with studies finding limited evidence that sequencing iteration before recursion substantially changes short-term performance [27]. Compared to prior studies, our work contributes a more activity-sensitive characterization by analyzing correctness and time-on-task separately across Use, Modify, and Create programming activities.

Taken together, these observations provide an empirical characterization of student performance with iteration and linear recursion at the end of CS1 under a widely used curricular structure. Rather than prescribing pedagogical changes, our findings delineate the magnitude and consistency of performance differences that subsequent computing courses may implicitly rely upon. More broadly, the results suggest that recursion difficulty manifests not only as lower correctness but also as increased variability and polarization, particularly as programming tasks require greater ownership and synthesis. This highlights the importance of examining how conceptual, representational, and cognitive demands interact with activity structure in introductory programming.

An important direction for future work is to disentangle the sources of these gaps, including the roles of instructional sequencing, programming language affordances, task characteristics, and extended exposure. Longitudinal studies could investigate whether the observed performance differences persist, diminish, or reverse as students develop more sophisticated mental models of recursion. It would also be valuable to examine how scaffolding strategies, alternative curricular trajectories, and opportunities for deliberate practice influence not only average performance but the distributional patterns observed here.

At a methodological level, the Use–Modify–Create framework proved instrumental in revealing both differences and similarities in how students perform with iteration and recursion. By structuring tasks along dimensions of agency and code ownership, the framework made it possible to observe patterns—such as the stability of correctness gaps alongside shifting distributional shapes and increasing polarization—that would be difficult to detect in undifferentiated task settings. More broadly, our findings point to the need for further development of analytic frameworks such as Use–Modify–Create that can describe and differentiate the specific forms of complexity, conceptual relations, and content knowledge involved in programming tasks. Refining such frameworks may enable more precise accounts of how difficulty emerges across activity types and support comparative studies that connect pedagogical design, cognitive processes, and learning outcomes across diverse computing curricula.

REFERENCES

- [1] Harold Abelson and Gerald Jay Sussman. 1996. *Structure and interpretation of computer programs*. The MIT Press.
- [2] ACM, IEEE Computer Society, and AAAI Joint Task Force. 2023. *Computer Science Curricula 2023*. CS2023 Report. Association for Computing Machinery & IEEE Computer Society, New York, NY. <https://doi.org/10.1145/3664191> Also available online via ACM Digital Library.
- [3] Heidi Goodrich Andrade. 2000. Using Rubrics to Promote Thinking and Learning. *Educational Leadership* 57, 5 (2000), 13–19.
- [4] Arooba Aqeel, Norman Peitek, Sven Apel, Jonas Mucke, and Janet Siegmund. 2021. Understanding Comprehension of Iterative and Recursive Programs with Remote Eye Tracking. In *Proceedings of the 32nd Annual Workshop of the Psychology of Programming Interest Group, PPIG 2021, Online, June 21–25, 2021*. Psychology of Programming Interest Group. <https://ppig.org/papers/2021-ppig-32nd-aqeel/>
- [5] David John Barnes, Michael Kölling, and James Gosling. 2006. *Objects First with Java: A practical introduction using BlueJ*. Pearson/Prentice Hall.
- [6] A. C. Benander, B. A. Benander, and H. Pu. 1996. Recursion vs. iteration: An empirical study of comprehension. *Journal of Systems and Software* 32, 1 (1996), 73–82.
- [7] Claus Brabrand, Sebastian Mateos Nicolajsen, Louise Meier Carlsen, and Michael E. Caspersen. [n. d.]. From Lecture-Based to Zipped Teaching Format. Manuscript in preparation.
- [8] H. Deitel and P.J. Deitel. 2013. *Java How to Program*. Pearson Education Limited. <https://books.google.dk/books?id=pCtTBwAAQBAJ>
- [9] Ramy Esteero, Mohammed Khan, Mohamed Mohamed, Larry Yueli Zhang, and Daniel Zingaro. 2018. Recursion or Iteration: Does it Matter What Students Choose?. In *Proceedings of the 49th ACM technical symposium on computer science education*. 1011–1016.
- [10] Dror G Feitelson. 2015. Using Students as Experimental Subjects in Software Engineering Research—A Review and Discussion of the Evidence. *arXiv preprint arXiv:1512.08409* (2015).
- [11] Gudmund Skovbjerg Frandsen and Michael I. Schwartzbach. 2006. A singular choice for multiple choice. *ACM SIGCSE Bull.* 38, 4 (2006), 34–38. <https://doi.org/10.1145/1189136.1189164>
- [12] Rita Garcia and Michelle Craig. 2025. 20 Years Later: A Replication Study on Teaching CS1 Concepts. *ACM Transactions on Computing Education* 25, 2, Article 22 (June 2025), 33 pages. <https://doi.org/10.1145/3730405>
- [13] Carlisle E George. 2000. EROSI—visualising recursion and discovering new errors. *ACM SIGCSE Bulletin* 32, 1 (2000), 305–309.
- [14] Tina Götschi, Ian Sanders, and Vashti Galpin. 2003. Mental models of recursion. In *Proceedings of the 34th SIGCSE technical symposium on Computer science education*. 346–350.
- [15] Pawel Grabarczyk, Sebastian Mateos Nicolajsen, and Claus Brabrand. 2022. On the Effect of Onboarding Computing Students without Programming-Confidence or -Experience. In *Koli Calling '22: 22nd Koli Calling International Conference on Computing Education Research, Koli 2022, Koli, Finland, November 17–20, 2022*, Ilkka Jormanainen and Andrew Petersen (Eds.). ACM, 18:1–18:8. <https://doi.org/10.1145/3564721.3564724>
- [16] Bruria Haberman and Haim Averbuch. 2002. The case of base cases: Why are they so difficult to recognize?. In *SIGCSE Bulletin*, Vol. 34. 84–88.
- [17] Colton Harper, Karima Mohammed, and Stephen Cooper. 2025. A Conceptual Metaphor Analysis of Recursion in a CS1 Course. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE TS 2025)*. Association for Computing Machinery, New York, NY, USA, 457–463. <https://doi.org/10.1145/3641554.3701949>
- [18] H. Kahney. 1983. What do novice programmers know about recursion. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 235–239.
- [19] Maria Kallia and Sue Sentance. 2017. Computing teachers’ perspectives on threshold concepts: Functions and procedural abstraction. In *Proceedings of the 12th workshop on primary and secondary computing education*. 15–24.
- [20] Claudius M Kessler and John R Anderson. 1986. Learning flow of control: Recursive and iterative procedures. *Human-Computer Interaction* 2, 2 (1986), 135–166.
- [21] Natalie Kiesler. 2022. Mental Models of Recursion: A Secondary Analysis of Novice Learners’ Steps and Errors in Java Exercises.. In *PPIG*. 226–240.
- [22] Irene Lee, Fred Martin, Jill Denner, Bob Coulter, Walter Allan, Jeri Erickson, Joyce Malyn-Smith, and Linda Werner. 2011. Computational Thinking for Youth in Practice. *ACM Inroads* 2, 1 (Feb. 2011), 32–37. <https://doi.org/10.1145/1929887.1929902>
- [23] J. Lewis, P. DePasquale, and J. Chase. 2020. *Java Foundations: Introduction to Program Design & Data Structures*. Pearson. <https://books.google.dk/books?id=BxoluWEACAAJ>
- [24] J. Lewis, D.E.D. John Lewis, W. Loftus, and C. Cocking. 2006. *Java Software Solutions for AP Computer Science A*. Pearson Education, Limited. <https://books.google.dk/books?id=q5PPAAAACAAJ>
- [25] Renée McCauley, Scott Grissom, Sue Fitzgerald, and Laurie Murphy. 2015. Teaching and learning recursive programming: a review of the research literature. *Computer Science Education* 25, 1 (2015), 37–66.
- [26] Renee McCauley, Brian Hanks, Sue Fitzgerald, and Laurie Murphy. 2015. Recursion vs. iteration: An empirical study of comprehension revisited. In *Proceedings of the 46th ACM technical symposium on computer science education*. 350–355.
- [27] Claudio Mirolo. 2012. Is iteration really easier to learn than recursion for CS1 students?. In *Proceedings of the ninth annual international conference on International computing education research*. 99–104.
- [28] Douglas C Montgomery. 2017. *Design and analysis of experiments*. John Wiley & sons.
- [29] John Murnane. 1991. Models of recursion. *Computers & Education* 16, 2 (1991), 197–201.
- [30] Sebastian Mateos Nicolajsen, Michael E. Caspersen, and Claus Brabrand. 2025. Circle of Life: Microworld Project at the End of CS1. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1, SIGCSE TS 2025, Pittsburgh, PA, USA, 26 February 2025 - 1 March 2025*, Jeffrey A. Stone, Timothy T. Yuen, Libby Shoop, Samuel A. Rebelsky, and James Prather (Eds.). ACM, 854–860. <https://doi.org/10.1145/3641554.3701808>
- [31] Morelli Ralph and Walde Ralph. 2017. *Java, Java, Java: Object-Oriented Problem Solving*. Prentice Hall.
- [32] S. Reges and M. Stepp. 2020. *Building Java Programs: A Back to Basics Approach*. Pearson. <https://books.google.dk/books?id=lp8qvgEACAAJ>
- [33] Saul Schleimer, Daniel S. Wilkerson, and Alex Aiken. 2003. Winnowing: local algorithms for document fingerprinting (*SIGMOD '03*). Association for Computing Machinery, New York, NY, USA, 76–85. <https://doi.org/10.1145/872757.872770>
- [34] Tamarisk Lurlyn Scholtz and Ian Sanders. 2010. Mental models of recursion: investigating students’ understanding of recursion. In *Proceedings of the fifteenth annual conference on Innovation and technology in computer science education*. 103–107.
- [35] Sue Sentance and Jane Waite. 2017. PRIMM: Exploring pedagogical approaches for teaching text-based programming in school. In *Proceedings of the 12th Workshop on Primary and Secondary Computing Education*. 113–114.
- [36] Vladimir Sulov. 2016. Iteration vs recursion in introduction to programming classes: An empirical study. *Cybernetics and Information Technologies* 16, 4 (2016), 63–72.
- [37] Susan Wiedenbeck. 1988. Learning recursion as a concept and as a programming technique. *ACM SIGCSE Bulletin* 20, 1 (1988), 275–278.